

L1: Intro to NLP. Documents & Knowledge. Document: any source of NLP text (books, articles, web). Text is unstructured and ambiguous. **Knowledge Representation:** structured, precise, actionable, domain-specific data a computer can use. NLP: converts unstructured documents → structured knowledge. Documents: humans read slowly, get tired, can't remember all. Knowledge: computers use quickly, don't tire, answer questions fast. **Definition:** NLP: subfield of linguistics, CS, and AI for analyzing/understanding human language and processing large amounts of natural language data. **Motivation:** Industry adoption (analyze customer feedback → improve sales). Accessibility (voice interfaces for people with disabilities). Career demand high, especially with LLMs. Research growth: NLP papers among highest since 2017. **AI / ML / DL NLP.**

- **AI:** umbrella – making computers act like humans
- **ML:** subset of AI – learns from data (classification, clustering, forecasting, association, anomaly detection)
- **DL:** subset of ML – deep neural networks
- **NLP:** intersects ML and DL
- **ML data:** table where rows = instances, columns = features, last column = class/decision. All values must be numeric. Text must be converted to numbers before ML/DL. **History:** 1950s: translation machines. 1960s: ELIZA chatbot (**rule-based**, not DL). 1970s: statistical models. 2013: Word2Vec. 2017: "Attention Is All You Need" – Transformer. Post-2017: LLMs (GPT, etc.). **Transformer variants:** Encoder-only (BERT), Decoder-only (GPT), Encoder-Decoder (T5). **Turing Test:** Judge asks questions to a human & computer. If judge can't distinguish → pass. Proposed by Alan Turing, 1950. Key insight: understanding and generating language ≈ intelligence.
- **NLU v.s. sentiment analysis, text classification** (spam, priority, category, language detection, document/topic)
- **NLG only:** speech-to-text, auto report generation (e.g., medical reports from patient data)
- **NLU+NLG:** translation, summarization, QA, chatbots

Speech pipeline: Voice → Signal Processing → Acoustic Model (signals → words) → NLP (words → text). Chatbots are **bidirectional**: speech→text (input) + text→speech (output). **Dialogue management** tracks conversation state and controls flow. **Sentiment analysis:** requires deeper semantic understanding than text classification. **Text summarization:** one of the hardest NLP tasks (coherence, redundancy, complexity, language-specific). **QA: Extractive** = locate/extract answer from text. **Generative** = generate answer from scratch. **Text classif. vs Sentiment:** classif. = shallow/keyword-based; sentiment = deep semantic understanding of emotion/attitude/trends. **LLMs** (e.g., ChatGPT): text generation + reinforcement learning, built on Transformer architecture. **Challenges in NLP.**

- 1. **Ambiguity:** lexical ("bank"), attachment ("saw man with telescope"), coreference ("she")
- 2. **Sparsity / Zipf's Law:** $f(w) \propto 1/r$ (frequency inversely proportional to rank); rare words are not outliers in NLP – can carry critical meaning (e.g., "fuzzy logic" appears once but is important). >1/3 of words occur only once
- 3. **Variation:** lexical ("gave the book to Tom") = "gave Tom the book", geographical (regional dialects), social (professor vs friend), stylistic, generational, cross-linguistic (English NLP ≠ French NLP)
- 4. **Common knowledge:** humans share implicit knowledge computers lack ("a man with a dog" vs "a dog with a man" – both valid, humans know which is natural; "Earth is round")
- 5. Volume, accents/slang, computation, security/privacy

Approaches to NLP: Heuristic/Rule-based (regex) → Machine Learning → Deep Learning (RNN, LSTM) → Transformers (2017+). Old techniques still useful for edge cases. **NLTK:** learning/prototyping. **spaCy:** industrial-strength. **Hugging Face:** pre-trained models & Transformers. **scikit-learn:** preprocessing, vectorization, ML models. **L2: Regex, Pipeline, Preprocess-**

a → ab, al
? 0 or 1 of preceding
+ 1 or more of preceding
* 0 or more of preceding
[] any one char inside
{ } negation (not in set)
^ start of string
\$ end of string
\ specific repetitions
\ escape metachar
\ alternation (or)
\ grouping (exact sequence)
Key: [abc] = any one of a,b,c. (abc) = exact sequence a,b,c in order. Ranges: '[a-z]', '[A-Z]', '[0-9]'. Negation: 'has dual meaning' – inside [!]: negation ('[A-Z] = NOT uppercase'); outside [!]: anchor ('[A-Z] = uppercase at start'). Examples: colour? → color/color. beg.a → begin/begun. \. \$ → literal period at end. **Char classes:** \s whitespace, \w word char (slides: [A-Za-z]: Python re: [A-Za-z0-9_]). \d digit, \b word boundary. Uppercase = negation. **Python re Library:** re.match: match at beginning only. re.search – first match anywhere. re.findall – list of all non-overlapping matches. re.sub – replace all matches. re.compile – compile into reusable pattern object. re.split – split string at pattern matches. re.finditer – iterator of match objects. Match objects: group() – matched text. .span() – (start, end) indices. **Regex Applications in NLP.** Text cleaning, tokenization, info retrieval, simple sentiment (count good vs bad words), language detection (pattern matching for language-specific chars). **NLP Dev Lifecycle.** 1. Understand problem & requirements 2. Data collection (large & relevant) 3. Text cleaning 4. Preprocessing 5. Feature extraction 6. Modeling 7. Evaluation 8. Deployment 9. Monitoring. Non-linear loop back from evaluation to preprocessing. Step 1: decision on evaluation right approach. Step 9: watch for model drift (new terminology/patterns over time degrade performance). **Garbage in, garbage out.** Always explore data first (like EDA in ML). **Preprocessing Pipeline.** Raw doc → **Tokenization** → **Noise Removal** → **Normalization** → clean tokens. **Why:** improve model performance, reduce dimensionality, standardize input from different sources (PDFs, web, text files). **Building Blocks of Language.** Phonemes (44 sounds) → Morphemes/Lexemes (smallest meaningful unit, e.g., "untangling" = "un" + "tangle" + "ing") → Syntax (grammar rules) → Context. **Corpus:** collection of docs. **Vocabulary:** set of unique words. Word: unit of language separated by options (punctuation). **Tokenization:** Divide text into tokens (not just words). Punctuation & contractions become separate tokens ("can't" → "can" + "'t"). Semicolons/punctuation are tokens but **not** words. Whitespace is **not** the only split criterion. **NLTK:** word.tokenize(), sent.tokenize(), regex.tokenize() (custom regex). **Vocabulary** ≠ tokens: vocab = set of unique words; tokens = all units including duplicates. **Noise Removal:** Remove: numbers, punctuation, **stop words** (179 in NLTK), URLs, HTML tags, handles, hash-tags (keep/remove depends on task). Keep emojis for sentiment (replace with word equivalents). Lowercase conversion. **Compound words** ("New York", "machine learning"): keep as one token using dictionary. **Stop words:** most frequent words carry least meaning. **Codes:** re.sub("^\d+", "", text). rm numbers. Remove punct: text.translate(str.maketrans("", string.punctuation)). [w for f in tokens if f not in stop_words] filter stop words. **Normalization.** **Stemming:** rule-based suffix stripping. Fast, may produce invalid words ("studies" → "studi", "helps" → "help"). Porter, Snowball (multilingual), Lancaster (aggressive). Useful for: classification, clustering, search engines/info retrieval. **Lemmaatization:** vocabulary + morphological analysis → **lemma** (root word). Always valid words ("studies" → "study", "better" → "good", "am/is/are" → "be"). Slower but more accurate. **NLTK WordNetLemmaizer, spaCy. spaCy: lemmatization only (not stemming).** Use lemmatization when accuracy > speed. Skip normalization for: poetry analysis, morphological analysis, social media analysis (variations convey emotion/attitude). **POS Tagging & NER.** POS: identifies noun, verb, adj, etc. pos.tag(tokens). Tags: NN (noun), NNS

(plural), VB (verb), VBZ (3rd person), JJ (adj), RB (adv), DT (det), IN (prep), PRP (pronoun). nltk.help.tagset() for full list. Why: interactive/semantic analysis improves downstream tasks (NER, parsing, translation). **NER:** identifies people, places, orgs, phone numbers, emails. ne_chunk(pos.tag(...)). Entity types: PERSON, GPE (geo-political), ORGANIZATION. Why: info extraction, search/indexing, identify which entity a sentiment targets. **L3: Feature Representation & Similarity. Feature Engineering.** Convert text → numerical table for ML. Equally important as preprocessing. On Kaggle, a single new feature can win a contest. **Data representations:** images = matrix (pixel intensity), speech = waveform (amplitude), text = **vectors**. **Evolution:** Frequency-based (statistical Word Embedding) → Transformer-based (LLMs). **Vector Space Model.** Each word/doc = vector of word counts. Similarity → nearby vectors. **Vectorization:** encoding text as integers → **feature vectors**. All techniques produce vectors; key difference = how well values reflect semantic meaning. **Norm:** $\|\vec{v}\| = \sqrt{v_1^2 + \dots + v_n^2}$ Dot product: $\vec{a} \cdot \vec{b} = \sum a_i b_i$ **Early Attempt: Linguistic Vectors.** Experts manually answered questions per word ("Is it male?", "Is it living?", "Can it talk?") → binary vectors. First attempt at vector space. Not scalable, language dependent. **One Hot Encoding.** Binary vector: 1 if token exists, 0 otherwise. Dim = vocab size (300K+ English words). Steps: tokenize → build vocab → assign unique IDs → binary vector. Pros: reversible, interpretable, preserves position. Cons: high dimensionality, sparse, no semantic relation between words. Ex: "This is an example" → This=[1,0,0,0], I=[0,1,0], am=[0,1,0,0], example=[0,0,0,1]. **Bag of Words (BoW).** Vectors of word frequencies. Ignores word order. Improved over OHE, "child makes dog happy" = "dog makes child happy" (same BoW). "John is quicker than Mary" = "Mary is quicker than John". Pros: simple, language independent, works for text classif/info retrieval. cv=CountVectorizer(); X=cv.fit_transform(corpus) – builds BoW matrix from corpus. Defaults: lowercase=True, token_pattern requires 2+ chars (drops single-char words like "I"). cv.get_feature_names_out() – returns vocab sorted alphabetically. X.toarray() – converts sparse matrix to dense. **Bag of N-grams.** counts frequency of n consecutive words. Introduces local context. n=1: unigram (e.g., "W"), n=2: bigram, n=3: trigram. Larger n = more context but more computation. **CountVec vs TF-IDF:** CountVec (n=2) bigrams, TF-IDF (n=1,2) – uni+bigrams. Ex: "I love NLP" bigrams: [I love, love NLP]. "I am learning NLP" trigram: "am learning NLP". **TF-IDF.** TF-IDF(t, d) = TFF(t, d) × IDF(t, d) = $\frac{\text{count of t in d}}{\text{total terms in d}}$

IDF(t) = $\log_{10} \left(\frac{\text{total docs}}{\text{docs with t}} \right)$ Rare words → higher weight (high IDF). Common words → lower weight (low IDF). **TF-IDF:** tfidf=vectorizer(doc); X=tfidf.fit_transform(docs) – builds TF-IDF weighted matrix from docs. Originated from info retrieval (before NLP). Not suited for small corpora: (IDF misleads). Zeros carry info (word absent = attribute). Ex: 1000 docs, 100-word doc, "Trump" appears 5x. TF = 5/100 = 0.05. If in 50 docs: IDF = log(1000/50) = 1.3. TF-IDF = 0.065. If in only 5 docs: IDF = log(1000/5) = 2.3. TF-IDF = 0.115 (rarer = higher weight). No polysemy: "bank" (financial) and "bank" (river) get same vector. **CountVec vs TF-IDF:** CountVec = whole numbers (raw counts); TF-IDF = real numbers (weighted). TF-IDF weights "milk" (rare) higher than "hot" (common) → smarter similarity. All frequency methods: no semantics, sparse, high dim, OOV problem. Still useful when frequency matters more than semantics (trade-off accuracy vs computation). **Text Similarity Metrics.** Applications: plagiarism detection (text overlap), text classification (text classification), text clustering (text clustering). **Benchmark to evaluate/compare representation techniques.** Jaccard: set overlap $J(A, B) = |A \cap B| / |A \cup B|$. **Hamming:** differing positions in equal-length strings. **Levenshtein:** min single-char edits (insert, delete, sub). "kitten" → "sitting" = 3 (k→s, e→i, insert g). "intention" → "execution" = 5. **Euclidean:** $d = \sqrt{\sum (a_i - b_i)^2}$. Measures straight-line distance (magnitude only, not direction). **Cosine Similarity** (most used in NLP): measures angle between vectors (magnitude + direction). $\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$ cos=1: identical. cos=0: no similarity. cos=-1: opposite. Ex: $\vec{A}=[1, 2, 1]$, $\vec{B}=[2, 1, 1]$. $\vec{A} \cdot \vec{B} = 5$, $\|\vec{A}\|=\|\vec{B}\|=\sqrt{6}$. $\cos = 5/6 \approx 0.833$ (high). Ex: "I love NLP" $\vec{a}=[1, 1, 0, 1]$ vs "I love you" $\vec{b}=[1, 1, 1, 0]$. $\cos = 2/3 \approx 0.667$. **As benchmark:** represent → compute cosine → high score = good representation. TF-IDF + cosine + BoW + word embeddings (word embeddings). **Choosing Representation.** Simple classif/info retrieval: TF-IDF. Semantic understanding → word embeddings. Deep contextual → Transformers. Always consider accuracy vs computation trade-off. **Ethical concerns:** false similarity/dissimilarity (e.g., plagiarism detection errors), term/topic bias (popular topics overweighted, minority topics underrepresented), language bias (English dominance on internet). **L4: Word Embedding.** Motivation: Count-based: sparse, high-dim, no semantics, OOV. Goal: dense vectors with semantic meaning. **Distributional hypothesis:** words in similar contexts are similar. Semantic field: words covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton). Synsets (synonyms), gloss (definition). **Hypernym:** broader term ("animal" → "dog"). **Hyponym:** specific term ("dog" → "animal"). Also: meronym (part-of), antonym (opposite). Synset: (car, automobile, motorcar). Holonym (whole): car→wheel. Troponym: "run" → "move". Entailment: "snore" → "sleep". Used for query expansion: query → keywords → WordNet synonyms → expanded search. Not computational, static, English-only. wordnet.synsets("car") – all synsets for word. .print_definition() gloss. .print_usage_examples(). **Word Embedding:** Maps words to dense vectors covering a domain (restaurant → food, cinema, wine, etc.). Embeddings capture word semantics & contextual relations. **WordNet:** Lexical database (Princeton).

from decoder, K, V from last encoder output. $\text{softmax}(Q \text{enc} K_{\text{enc}}^T / \sqrt{d_k}) V_{\text{enc}}$. Decoder query searches encoder keys to retrieve values. "Given my decoder state, what in history to attend to?" **Prediction:** decoder output $\rightarrow$ Linear (vocab-size) **Softmax** $\rightarrow$ argmax $\rightarrow$ query lookup $\rightarrow$ word. **Transformer families:** **Encoder-only:** BERT, DistilBERT, RoBERTa, XLM, ALBERT, ELECTRA, DeBERTa. **Encoder-Decoder:** T5, BART, GPT-2, GPT-3, GPT-4. **Decoder-only:** GPT, GPT-2, GPT-3, CTRL, GPT-Neo, GPT-J. **Hugging Face:** pre-trained models, datasets, tokenizers, for transformers import pipeline classifier=pipeline(("text-classification")) # "positive", "negative", "neutral", "spam", "toxic", "hate", "insult", "threat", "sexual", "(AFC), "pipeline"("question-answering") # extractive QA, pipeline("summarization") # text summary. **Always read model cards** (training data, params, limits). **HF app ex (Bumblebee/Amazon complaint):** text-classify $\rightarrow$ NEGATIVE 0.90, NER $\rightarrow$ Amazon(ORG), Optimus Prime(MISC), Germany(LOC), Megatron(MISC), Bumblebee(PER), QA "What does customer want?" $\rightarrow$ "an exchange of Megatron". Summary: condensed complaint. **Challenges:** English-centric, huge data needed for fine-tune, long docs struggle (O(N) attn cost), opacity (billions params), bias from data, model overfitting, no interpretability, no model control, no model-level learning rate schedule (warmup). Paper used Adam, label smoothing 0.1, dropout 0.1. **L10: BERT & Question Answering.** Model timeline: Jun'17 Transformer, Jun'18 GPT, Oct'18 BERT, Feb'19 GPT-2, Oct'19 T5, May'20 GPT-3, Sep'21 FLAN, Mar'22 GPT-3.5/InstructGPT, Nov'22 ChatGPT, Feb'23 LLaMA, Mar'23 GPT-4, Mar'24 GPT-4o, Apr'24 LLaMA-3.1 (405B), Dec'24 o1 DeepSeek-V3, Jan'25 DeepSeek-R1, BERT (Bidirectional Encoder Representations from Transformers, Devlin, 2018, Google). Encoder-only branch: trained on Wikipedia (2.5B) and books (800M) (L11, L12). **Input embedding:** fixed vector. Textual search engine: "bank" (money) differ. Pre-BERT (Word2Vec): **inverted** vector. Google Search big win post-BERT. **Architectures:** BERT-base: 12 layers, hidden 768, 12 heads, 110M params. BERT-large: 24 layers, hidden 1024, 16 heads, 340M params. Vs original transformer (6 layers, 512-dim, 8 heads). **More is better.** Hidden 768 fixed per token (no choice like Word2Vec). **BERT = stacked encoders.** Main block unchanged vs transformer; only input/output change. **Max input = 5 tokens** (not words, incl specials). Fed all at once. **Special tokens:** [CLS] = [CLS] [MASK] = [MASK] [SEP] = [SEP] [EOS] = [EOS] [EOT] = [EOT] [MASK] = [MASK] (training). **Preprocess:** tokenize (WordPiece) $\rightarrow$ add CLS/SEP/MASK $\rightarrow$ map IDs $\rightarrow$ + positional + segment encoding. **WordPiece:** learned tokenization. Vocab = 30,522. OOV $\rightarrow$ sub-pieces with ## prefix: "playing" $\rightarrow$ "play" + "##ing". Reconstruct by merging ##-prefixed pieces. "unbelievable" $\rightarrow$ "un" + "##believe" + "##able". **Position encoding** (BERT): learned, not fixed sin/cos like transformer. Part of backprop. **Segment encoding:** 0=sent A (question), 1=first B (answer), 2=second B (answer), 3=third B (answer). **Encoding (CLS) dog is like [SEP] he likes play ##ing [SEP]** $\rightarrow$ (past 5 tokens = seg A, last 5 = seg B. **Outputs:** 1) **last_hidden_state**: per-token contextual embed, shape (batch, seg, 768). 2) **pooler_output** = CLS embed (batch, 768). CLS summarizes whole sentence (bidirectional multi-head attn). Use CLS for classification; per-token for NER/QA. 2 BERT benefits: per-token embed (like Word2Vec) + CLS classif head. (new capability). **Training (simultaneous):** (1) MLM (Masked LM): mask 15% tokens. Of those: 80% [MASK], 10% random, 10% unchanged. Prevents model from only learning [MASK] pattern. LM head w/ softmax over vocab predicts masked. CE loss only at masked positions. $\mathcal{L}_{MLM} = \sum \log y_{w_i}$. **CLS:** [CLS] Deep [MASK] is very Power [SEP] $\rightarrow$ predict "Learning". **Ex:** "So [mask] and [mask] for all apricot fish" $\rightarrow$ "So long and thanks for all the fish". (2)

NSP (Next Sentence Prediction): "is B really next of A?" Feed [CLS]A[SEP]B[SEP]. CLS → NSP head (2-class softmax) → IsNext/NotNext. $\text{Loss}_{\text{NSP}} = \log y_i$. [EX]: Man went to store" → "He bought milk" = IsNext. "Man went to store" → "Penguins flightless" = NotNext.

$L_{\text{total}} = L_{\text{MLM}} + \text{NSP}$.

Pre-train + Fine-tune paradigm: pre-train once (MLM+NSP huge data) → freeze layers → add task-specific head → fine-tune on task data. **Classification w/BERT**: sentence → BERT → CLS → softmax → class probs. Ignore per-token outputs. **Token classification**: token → BERT → token embedding (Dense) → Linear layer → vocab size classes.

Inference: tokens from pretrained ("bert-base-uncased") model=BertModel.from_pretrained("bert-base-uncased"). input=tok(sent, padding=True, truncation=True, return_tensors='pt') → returns input_ids, attention_mask, token_type_ids with torch.no_grad(). out=model(*inputs). no_grad=freeze embed. Use for inference. out.last_hidden_state (.seq_len), out.pooler_output (1,768). output.hidden_states=True → 13 layers (12 enc + input embed). Same "bank" in diff contexts: cos ≈ 0.4 (proves contextuality).

Google released 6 checkpoints: BERT-Base/Large Cased & Uncased (12L/24L, 768/1024, 12/16 heads, 96 dim.). Multilingual Cased (11 lang.), Multilingual uncased (11 lang.). Variants: DistilBERT ~7% smaller, 40% faster. RoBERTa, ALBERT.

ALBERT: param sharing. ELECTRA: replaced token-detection objective. SpanBERT: span masking. TinyBERT: tiny. SciBERT/BioBERT/ClinicalBERT: domain. Cased vs Uncased (uncased=lowercased for generic; cased for NER). BERT is open source (vs GPT). **QA System**

3 dimensions: (1) info source, (2) question type, (3) answer type. **Info sources**: structured (SQL/NoSQL DB), unstructured (web, papers, manuals, books), conversational (customer log forums, social). **Question types**: factoid (concrete), open-domain (in the field), closed-domain (specific assignment 2), complex narrative (between-the-lines). **Answer types**: extractive (single word or phrase), generative (generate full paragraph) (DB), yes/no.

modules - Paradigms:

- (1) Extractive QA (Seq2QAD).
- (2) Knowledge-based.
- (3) Hybrid (IBM Watson modules: 4 modules – Q processing, candidate gen, scoring, merging-ranking, Won Jeopardy Feb 16, 2011).
- (4) Generative (LLMs).
- (5) RAG (use for project).

Watson modules: M1 Q Processing (focus, lex type, classify, parse, NER, relation, coref). M2 Candidate Gen (text retrieval + DB/Pipe/Freebase). M3 Scoring (text, time, ansver type, space evidence). M4 Merge+rank (logistic regression). SQuAD (Stanford 2016): 100K triples (passage, Q, answer). Wikipedia passages 100–150 words. Crowd-sourced. Answer = text span inside passage. Ex (meteorology): "Q causes precipitation to fall?" → "gravity". Q form besides drizzle/rain/snow/sleet/hail/fog? → "any weather condition." SQuAD v1.1 test set: 100k questions over 500 articles. Match EM (EM): 0/1 per example. F1: token-level overlap (harm mean P/R). Dev/test: 3 gold answers/Q, take max across folds, avg over examples. Ex: pred="Left Graz and seved" vs golds. EM=0. F1=max(0.67, 0.67, 0.61)=0.67. BiDAF (LSTM era, 2016): Bidirectional Attention Flow. Char+WOrd+Phrase embed → C2Q+C2Q attn → modeling LSTM → Start (Dense+softmax), End (LSTM+softmax). F1=81.2 on SQuAD 1. Post-BERT SQuAD 2.0: BERT-base F1=88.5, BERT-large=90.9, XLNet=94.5, RoBERTa=94.6, ALBERT=94.8 (BiDAF=77.3). **Extractive QA w/ BERT**: infer [CLS] q [SEP] Passage [SEP]. Segments A_i=passage. Two heads: p_start(i) = softmax(w_start H_i), p_end(i) = softmax(w_end H_i). Single h_start, end weights. h_start = log P_start(s*) / sum_j log P_start(j*). Long passages: start_idx (overlapping window), tokenizer(q). length=500. Only truncate at stride=25, return=overflowing_tokens True, return_offsets_mapping=True) only truncate context, preserve char offsets. Code: from transformers import AutoTokenizer, AutoModelForQuestionAnswering start_idx=argsmax(start_logits); end_idx=argsmax(end_logits) ans=tkenizer.decode(input_ids[start_idx:end_idx+1]).

Retriever Reader: removes need to provide context. Only Q given. Flow: Q → Retriever (+docstore) → top-K docs → (post-proc rank) → Reader (QA model) → answer span → (post-proc) → User. Retriever: TFIDF, BM25 (statistical), L2MSimilarity, DenseEmbedder, CosineSimilarity, ReciprocalRank Fusion (RRF). Milvus. State-of-the-art embeddings limited to some stores; embedding retrievers work with all DPR (Dense Passage Retrieval): dual-encoder. Q encoder + P encoder, trained end-to-end w/ 1 loss. Dot product similarity = doc score. At inference: Q → Q-enc → find top similar P-vectors. Retriever X Store compat: TF-IDF works w/ in-memory, ES (not FAISS/Milvus). BM25 w/ ES only. Embedding/DPR work w/ all 4. Embedding backbones for retrieval: Word2Vec, GloVe w/ BERT. Haystack (deepest): open-source retriever-reader framework. From haystack.document_store.elasticsearch.ElasticsearchDocumentStore(using Elasticsearch) pipe=TextIndexerPipeline(retriever=retriever=FABRETRIEVER(deepest_roberta_base_squadv2)) pipe.extractive_q_pipeline(retriever=pipeline.run(query="", params={"Retriver":{"top_k":10}, "Reader":{"top_k":5})}). Alternatives: DeepPavlov, DRQA. Other datasets: WikiQA, TRECQA, NewsQA, Google NQA, TriviaQA, HotpotQA, MS MARCO. RAG extends Retriever-Reader with generative (LLM) reader.

L12: LLMs & RAG. LLM evolution: 2019 T5/GPT-3; 2021 many (FLAN, LaMDA, InstructGPT, Chinchilla, Gopher, Ernie 3.0); 2022 OPT/BLMO/PAI/Mistral/DeepSeek/ChatGPT-3. Pythia/Vicuna/palM2/Falcon/LLaMA/GPT-4o/BloomAI/Plama/FlanT5/ChatGPT-3. Gemma3 (Google), OpenAI GPT-4o, Claude3 Opus, Gemini 1.5 Pro, Mistral 3.

LLMs: ML predict next word. LLM massive w/ transformer arch billions of parameters + performs many NLP tasks (classify, summar, QA, translation). Components identifying LLM: (1) tokenizer, (2) parameters (weights), (3) context window, (4) architecture (transformer; Mamba emerging). Parameter = weight. Tiny NN 3→4→4→1 = 41 params (12w+4b, 16w+4b, 4w+1b). LLMs: millions. More params = more power. Size table: BERT-L 340M (1.3GB FP32), GPT-4o ~200TB (350GB). LLaMa 13B/70B (26/140GB). Bloom 176B (352GB). Mistral 7B (14GB). Mixtral 8x7B 56B (112GB). PaLM 540B (~1TB). Consider beyond disk training data size, fine-tune overhead. Tokenizer: diff per LLM. Same text → diff token counts. Tokens are pairs of tokens → fine-tune. Reward Model: humans rate AI responses. Window = max tokens processed at once = prompt + conversation history + generated answer tokens. Gemma2/Mistral 32K. LLaMa3.1 128K. Claude3.5 200K. GPT-4.1 1M. Gemini 1.5 Pro 1-2M (~3000 pages). Larger = long conv, continuous reasoning. Types by training:

- (1) General-purpose (GPT).
- (2) Instructional (AllenNLP) – step-by-step.
- (3) Conversational (DialogGPT).
- (4) Domain-specific (BioBERT biomedical) – assignment. ChatGPT is NOT an LLM; fine-tuned GPT-4o for conversation. GPT closed-source since GPT-3. ChatGPT training pipeline: (1) collect docs, (2) chunk, (3) base on hugoboss-stage-SFT (prompt, human-labeled response), self-critique Redwood-Medial humandata, can make good choices.

D(D > C > A) = B → train RM to predict human rank. Stage 3 RLHF: policy generates → RM scores → update policy via PPO (Proximal Policy Optimization). GPT-4 = multimodal (text+img+audio). Constitutional AI (CAI): Claude, GPT-4o. Replaces human feedback w/ AI self-eval Principles: helpful, harmless, ethical, virtuous. SL stage: generate responses → AI critiques & revises → SFT on revisions. RL stage: AI generates preference pairs → train preference model → fine-tune via RLAI/RL (from AI Feedback). Reduces human labor, cost, time (time saving). RLHF: Humans rating every output, costly/slow. Can only initial principles, self-critique led-back to fine-tune. Fine-tuning data can come from chat logs + output report. Trained on 100 nat langs, 20 prog langs, sci papers, math. Match model to domain. LLM Limitations: (1) Training cutoff: static mitigated by internet search (but noisy, unreliable). GPT-5: Sep 30 2024 cutoff, 400K ctx, \$1.25/\$10 per M tokens. (2) Limited context: even 2M has "lost in the middle" – good at start/end, weak middle. Long ctx can degrade quality. (3) Consistency/hallucination: same prompt may vary. Galactica pulled for confident false claims. (4) No big specialized LLMs: limited audience → less investment in domain-specific at scale. (5) OpaCity: billions of params hard to interpret. (6) Reproducibility/reproducibility: poor reproducibility. (7) Reliability: inconsistent results. (8) Compute: inference needs more than weight storage. RAG (Retrieval-Augment-Generation): mitigates cutoff, grounds answers in your docs. Workaround: augment LLM w/ extra documents. Why RAG: domain-specific accuracy, frequent updates (re-index, no retrain), traceability (trace answer to source), controllable cost (swap store, no retrain), privacy (sensitive docs stay local). Components: Retriever (external, info retrieval) + Reader (LLM, text generation). Pipeline: (1) Collect docs, (2) Chunk, (3) Embed chunks, (4) Store in vector DB, (5) User Q, (6) Embed Q, compare via similarity, (7) Top-K chunks = ranked context. (8) LLM generates answer from prompt+context, (9) return to user. Similarity metrics: Euclidean, cosine, KNN, inner products. Stages: parsing (corpus→chunks), indexing (chunks→embeddings), Semantic Search (vector DB, cosine sim), Retrieval (retrieve relevant chunks), Ranking (ordering by similarity not exact match (vs classic DB: structured, exact match)). Key traits: indexing technique (maindifferentiator), retrieval speed, similarity metric/purpose. Dedicated openness: Chromadb (recommended), Marqo, Vespa, Qdrant, LanceDB, Milvus. Commercial: Pinecone, Weaviate. Extensions: pgvector (Postgres), Elasticsearch, Redis, OpenSearch.

Embedding: FAISS. **Cloma code:** import chromadb; from chromadb.utils import embedding_functions
Embedding functions: SentenceTransformerEmbeddingFunction(model_name="all-MiniLM-L6-v2")
coll.query.create_collection("docs", embedding_function=emb) coll.add(documents=[...], ids=[...])
coll.query(query, texts=[...], n_results=1). **Chunking:** Fixed size: chars/tokens per chunk.
Fast, easy. Border problem (answer cut in half), no semantics. Overlap reduces border issues.
Semantic: split on natural boundaries (sentence/paragraph). Meaningful. variable
chunk overlap: "Llama 2 was tra/ined on 40% mo/rt" / "e data than Lla/ma 1". tokens/5
"Llama 2 was tra/ined on 40% mo/rt" / "e data than Lla/ma 1". tokens/5: "Llama 2 was
**trained on"/"40% more data than Llama"/"1", tokens/5 overlap 1: "., on"/"40% more data than
"/"than Llama 1". Prompt template (RAG): "answer based only on context: {context}
--- Q: {question}?". Curly = var, filled at query time. **Quality metrics (RAG eval):**
Rank=Model s. Relevance=human. May differ. **NDCG@K:** $DCG@K = \sum_{i=1}^K \frac{\log_2(l_i/(i+1))}{l_i}$
IR@K: rel at pos. **IDCG=DCG w/ rel sorted desc.** **NDCG = DCG/IDCG** in [0, 1]. **E**
K=5, rel=[1,3,2,0,1]: DCG=1+0.5+1.89+1.0+0.4+0.39=4.28. **Ideal=[3,2,1,0]=IDCG=5.19**
Recall@K=0.82. RR@K: 1/rank of first relevant. **Recall@K:** $\text{rel in top } K / \text{rel}$. **Precision@K:** $\text{rel in top } K / \text{rel}$
Temp controls determination.
LLMs: L1: greedy; L2: beam; Q2: beam; Q3: beam; Q4: beam; Q5: beam; Q6: beam; Q7: beam; Q8: beam; Q9: beam; Q10: beam; Q11: beam; Q12: beam; Q13: beam; Q14: beam; Q15: beam; Q16: beam; Q17: beam; Q18: beam; Q19: beam; Q20: beam; Q21: beam; Q22: beam; Q23: beam; Q24: beam; Q25: beam; Q26: beam; Q27: beam; Q28: beam; Q29: beam; Q30: beam; Q31: beam; Q32: beam; Q33: beam; Q34: beam; Q35: beam; Q36: beam; Q37: beam; Q38: beam; Q39: beam; Q40: beam; Q41: beam; Q42: beam; Q43: beam; Q44: beam; Q45: beam; Q46: beam; Q47: beam; Q48: beam; Q49: beam; Q50: beam; Q51: beam; Q52: beam; Q53: beam; Q54: beam; Q55: beam; Q56: beam; Q57: beam; Q58: beam; Q59: beam; Q60: beam; Q61: beam; Q62: beam; Q63: beam; Q64: beam; Q65: beam; Q66: beam; Q67: beam; Q68: beam; Q69: beam; Q70: beam; Q71: beam; Q72: beam; Q73: beam; Q74: beam; Q75: beam; Q76: beam; Q77: beam; Q78: beam; Q79: beam; Q80: beam; Q81: beam; Q82: beam; Q83: beam; Q84: beam; Q85: beam; Q86: beam; Q87: beam; Q88: beam; Q89: beam; Q90: beam; Q91: beam; Q92: beam; Q93: beam; Q94: beam; Q95: beam; Q96: beam; Q97: beam; Q98: beam; Q99: beam; Q100: beam; Q101: beam; Q102: beam; Q103: beam; Q104: beam; Q105: beam; Q106: beam; Q107: beam; Q108: beam; Q109: beam; Q110: beam; Q111: beam; Q112: beam; Q113: beam; Q114: beam; Q115: beam; Q116: beam; Q117: beam; Q118: beam; Q119: beam; Q120: beam; Q121: beam; Q122: beam; Q123: beam; Q124: beam; Q125: beam; Q126: beam; Q127: beam; Q128: beam; Q129: beam; Q130: beam; Q131: beam; Q132: beam; Q133: beam; Q134: beam; Q135: beam; Q136: beam; Q137: beam; Q138: beam; Q139: beam; Q140: beam; Q141: beam; Q142: beam; Q143: beam; Q144: beam; Q145: beam; Q146: beam; Q147: beam; Q148: beam; Q149: beam; Q150: beam; Q151: beam; Q152: beam; Q153: beam; Q154: beam; Q155: beam; Q156: beam; Q157: beam; Q158: beam; Q159: beam; Q160: beam; Q161: beam; Q162: beam; Q163: beam; Q164: beam; Q165: beam; Q166: beam; Q167: beam; Q168: beam; Q169: beam; Q170: beam; Q171: beam; Q172: beam; Q173: beam; Q174: beam; Q175: beam; Q176: beam; Q177: beam; Q178: beam; Q179: beam; Q180: beam; Q181: beam; Q182: beam; Q183: beam; Q184: beam; Q185: beam; Q186: beam; Q187: beam; Q188: beam; Q189: beam; Q190: beam; Q191: beam; Q192: beam; Q193: beam; Q194: beam; Q195: beam; Q196: beam; Q197: beam; Q198: beam; Q199: beam; Q200: beam; Q201: beam; Q202: beam; Q203: beam; Q204: beam; Q205: beam; Q206: beam; Q207: beam; Q208: beam; Q209: beam; Q210: beam; Q211: beam; Q212: beam; Q213: beam; Q214: beam; Q215: beam; Q216: beam; Q217: beam; Q218: beam; Q219: beam; Q220: beam; Q221: beam; Q222: beam; Q223: beam; Q224: beam; Q225: beam; Q226: beam; Q227: beam; Q228: beam; Q229: beam; Q230: beam; Q231: beam; Q232: beam; Q233: beam; Q234: beam; Q235: beam; Q236: beam; Q237: beam; Q238: beam; Q239: beam; Q240: beam; Q241: beam; Q242: beam; Q243: beam; Q244: beam; Q245: beam; Q246: beam; Q247: beam; Q248: beam; Q249: beam; Q250: beam; Q251: beam; Q252: beam; Q253: beam; Q254: beam; Q255: beam; Q256: beam; Q257: beam; Q258: beam; Q259: beam; Q260: beam; Q261: beam; Q262: beam; Q263: beam; Q264: beam; Q265: beam; Q266: beam; Q267: beam; Q268: beam; Q269: beam; Q270: beam; Q271: beam; Q272: beam; Q273: beam; Q274: beam; Q275: beam; Q276: beam; Q277: beam; Q278: beam; Q279: beam; Q280: beam; Q281: beam; Q282: beam; Q283: beam; Q284: beam; Q285: beam; Q286: beam; Q287: beam; Q288: beam; Q289: beam; Q290: beam; Q291: beam; Q292: beam; Q293: beam; Q294: beam; Q295: beam; Q296: beam; Q297: beam; Q298: beam; Q299: beam; Q300: beam; Q301: beam; Q302: beam; Q303: beam; Q304: beam; Q305: beam; Q306: beam; Q307: beam; Q308: beam; Q309: beam; Q310: beam; Q311: beam; Q312: beam; Q313: beam; Q314: beam; Q315: beam; Q316: beam; Q317: beam; Q318: beam; Q319: beam; Q320: beam; Q321: beam; Q322: beam; Q323: beam; Q324: beam; Q325: beam; Q326: beam; Q327: beam; Q328: beam; Q329: beam; Q330: beam; Q331: beam; Q332: beam; Q333: beam; Q334: beam; Q335: beam; Q336: beam; Q337: beam; Q338: beam; Q339: beam; Q340: beam; Q341: beam; Q342: beam; Q343: beam; Q344: beam; Q345: beam; Q346: beam; Q347: beam; Q348: beam; Q349: beam; Q350: beam; Q351: beam; Q352: beam; Q353: beam; Q354: beam; Q355: beam; Q356: beam; Q357: beam; Q358: beam; Q359: beam; Q360: beam; Q361: beam; Q362: beam; Q363: beam; Q364: beam; Q365: beam; Q366: beam; Q367: beam; Q368: beam; Q369: beam; Q370: beam; Q371: beam; Q372: beam; Q373: beam; Q374: beam; Q375: beam; Q376: beam; Q377: beam; Q378: beam; Q379: beam; Q380: beam; Q381: beam; Q382: beam; Q383: beam; Q384: beam; Q385: beam; Q386: beam; Q387: beam; Q388: beam; Q389: beam; Q390: beam; Q391: beam; Q392: beam; Q393: beam; Q394: beam; Q395: beam; Q396: beam; Q397: beam; Q398: beam; Q399: beam; Q400: beam; Q401: beam; Q402: beam; Q403: beam; Q404: beam; Q405: beam; Q406: beam; Q407: beam; Q408: beam; Q409: beam; Q410: beam; Q411: beam; Q412: beam; Q413: beam; Q414: beam; Q415: beam; Q416: beam; Q417: beam; Q418: beam; Q419: beam; Q420: beam; Q421: beam; Q422: beam; Q423: beam; Q424: beam; Q425: beam; Q426: beam; Q427: beam; Q428: beam; Q429: beam; Q430: beam; Q431: beam; Q432: beam; Q433: beam; Q434: beam; Q435: beam; Q436: beam; Q437: beam; Q438: beam; Q439: beam; Q440: beam; Q441: beam; Q442: beam; Q443: beam; Q444: beam; Q445: beam; Q446: beam; Q447: beam; Q448: beam; Q449: beam; Q450: beam; Q451: beam; Q452: beam; Q453: beam; Q454: beam; Q455: beam; Q456: beam; Q457: beam; Q458: beam; Q459: beam; Q460: beam; Q461: beam; Q462: beam; Q463: beam; Q464: beam; Q465: beam; Q466: beam; Q467: beam; Q468: beam; Q469: beam; Q470: beam; Q471: beam; Q472: beam; Q473: beam; Q474: beam; Q475: beam; Q476: beam; Q477: beam; Q478: beam; Q479: beam; Q480: beam; Q481: beam; Q482: beam; Q483: beam; Q484: beam; Q485: beam; Q486: beam; Q487: beam; Q488: beam; Q489: beam; Q490: beam; Q491: beam; Q492: beam; Q493: beam; Q494: beam; Q495: beam; Q496: beam; Q497: beam; Q498: beam; Q4**